DUOC UC · DESARROLLO ORIENTADO A OBJETOS II (PRY2203)

Guía de Ejercicios Resueltos

Semana 2 — Diseñando Jerarquías con Clases Abstractas

Experiencia 1: Aplicando principios avanzados de la POO

IL2. Diseña clases abstractas con sus métodos en jerarquías, para reutilizar comportamientos en clases derivadas.

Material de apoyo elaborado para la sesión sincrónica N.º 2

Profesor: Miguel Puebla

Introducción

Esta guía reúne **cinco ejercicios resueltos y comentados** que ponen en práctica los ejes trabajados durante la semana 2 de la Experiencia 1: clases abstractas, métodos abstractos y concretos, herencia y jerarquías, y el patrón de diseño *Template Method*. A diferencia del ejemplo usado en clases (sistema de transporte público), cada ejercicio aquí presentado utiliza un **contexto distinto**, de modo que puedas verificar que comprendiste el concepto y no solo memorizaste el ejemplo original.

> **Resultado de aprendizaje (RA1):** Desarrolla soluciones utilizando el paradigma de Programación Orientada a Objetos, focalizadas en la reutilización de código, el desacoplamiento, la escalabilidad y la mantenibilidad del software.
>
> **Indicador de logro (IL2):** Diseña clases abstractas con sus métodos en jerarquías, para reutilizar comportamientos en clases derivadas.

¿Qué encontrarás en cada ejercicio?

1. **Clases abstractas en Java** — Jerarquía de figuras geométricas
2. **Métodos abstractos y concretos** — Sistema de cálculo de sueldos
3. **Herencia y jerarquías con clases abstractas** — Instrumentos musicales
4. **Patrón de diseño Template Method** — Preparación de bebidas en una cafetería
5. **Aplicación integradora** — Reproductor de contenido de una plataforma de streaming

Cada ejercicio incluye: enunciado, diagrama de la jerarquía, solución completa en Java, clase `Main` de prueba, salida esperada en consola y una explicación de los conceptos aplicados.

1 Clases abstractas en Java

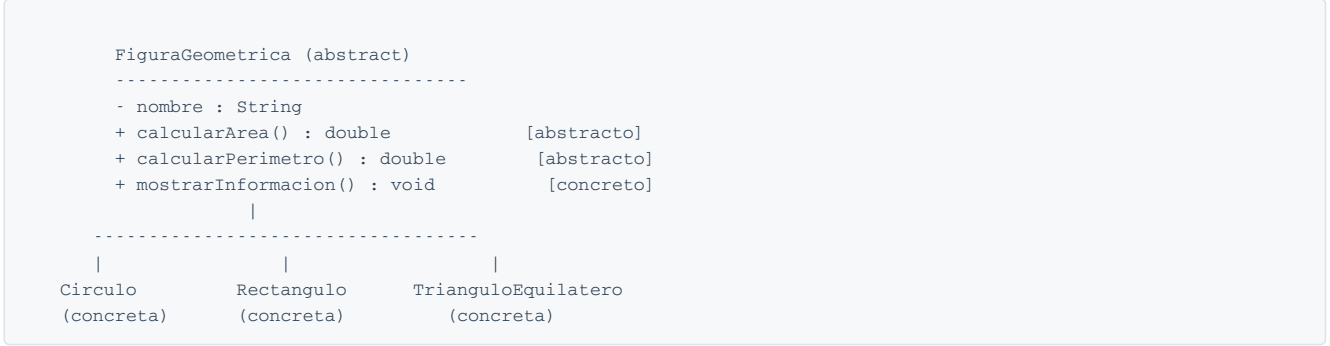
Eje 1

Enunciado: Jerarquía de figuras geométricas

Una empresa de diseño necesita un sistema que calcule el **área** y el **perímetro** de distintas figuras geométricas: círculo, rectángulo y triángulo equilátero. Todas las figuras comparten un nombre y la capacidad de mostrar su información en pantalla, pero cada una calcula su área y perímetro de forma distinta.

Se pide: diseñar una clase abstracta `FiguraGeometrica` que declare los métodos abstractos `calcularArea()` y `calcularPerimetro()`, y un método concreto `mostrarInformacion()` que sea reutilizado por todas las subclases.

Diagrama de la jerarquía



FiguraGeometrica.java

```
public abstract class FiguraGeometrica {

    protected String nombre;

    public FiguraGeometrica(String nombre) {
        this.nombre = nombre;
    }

    // Métodos abstractos: cada figura los implementa a su manera
    public abstract double calcularArea();
    public abstract double calcularPerimetro();

    // Método concreto: se reutiliza en todas las subclases
    public void mostrarInformacion() {
        System.out.println("Figura: " + nombre);
        System.out.printf("  Área: %.2f%n", calcularArea());
        System.out.printf("  Perímetro: %.2f%n", calcularPerimetro());
    }
}
```

Circulo.java

```
public class Circulo extends FiguraGeometrica {

    private double radio;

    public Circulo(double radio) {
        super("Círculo");
        this.radio = radio;
    }

    @Override
    public double calcularArea() {
        return Math.PI * radio * radio;
    }

    @Override
    public double calcularPerimetro() {
        return 2 * Math.PI * radio;
    }
}
```

Rectangulo.java

```
public class Rectangulo extends FiguraGeometrica {

    private double base;
    private double altura;

    public Rectangulo(double base, double altura) {
        super("Rectángulo");
        this.base = base;
        this.altura = altura;
```

```
    }

    @Override
    public double calcularArea() {
        return base * altura;
    }

    @Override
    public double calcularPerimetro() {
        return 2 * (base + altura);
    }
}
```

TrianguloEquilatero.java

```
public class TrianguloEquilatero extends FiguraGeometrica {

    private double lado;

    public TrianguloEquilatero(double lado) {
        super("Triángulo Equilátero");
        this.lado = lado;
    }

    @Override
    public double calcularArea() {
        return (Math.sqrt(3) / 4) * lado * lado;
    }

    @Override
    public double calcularPerimetro() {
        return lado * 3;
    }
}
```

Main.java

```
public class Main {
    public static void main(String[] args) {
        FiguraGeometrica[] figuras = {
            new Circulo(5),
            new Rectangulo(4, 6),
            new TrianguloEquilatero(5)
        };

        for (FiguraGeometrica figura : figuras) {
            figura.mostrarInformacion();
            System.out.println();
        }
    }
}
```

Salida esperada en consola

```
Figura: Círculo
  Área: 78.54
  Perímetro: 31.42

Figura: Rectángulo
  Área: 24.00
  Perímetro: 20.00

Figura: Triángulo Equilátero
  Área: 10.83
  Perímetro: 15.00
```

Explicación de los conceptos aplicados

- **FiguraGeometrica** no puede ser instanciada directamente porque es `abstract`; solo sirve como plantilla base.
- Los métodos `calcularArea()` y `calcularPerimetro()` quedan sin cuerpo porque cada figura los calcula de forma distinta: por eso se declaran `abstract`.
- `mostrarInformacion()` es un método **concreto**: su código es idéntico para todas las figuras, así que se define una sola vez y se reutiliza sin sobrescribir.
- Gracias al polimorfismo, el arreglo `FiguraGeometrica[]` puede almacenar objetos de distintas subclases y ejecutar el método correcto para cada una.

2 Métodos abstractos y concretos

Eje 2

Enunciado: Sistema de cálculo de sueldos

Una empresa contrata dos tipos de trabajadores: **empleados fijos**, que reciben un sueldo base más un bono mensual, y **empleados por horas**, cuyo sueldo depende de las horas trabajadas y el valor de la hora. Todos los empleados comparten un nombre y deben poder mostrar sus datos en pantalla, pero el cálculo del sueldo cambia según el tipo de contrato.

Se pide: definir una clase abstracta `Empleado` con un método abstracto `calcularSueldo()` y un método concreto `mostrarDatos()` que reutilice ese cálculo.

Diagrama de la jerarquía



Empleado.java

```
public abstract class Empleado {

    protected String nombre;
    protected double sueldoBase;

    public Empleado(String nombre, double sueldoBase) {
        this.nombre = nombre;
        this.sueldoBase = sueldoBase;
    }

    // Cada tipo de empleado calcula su sueldo de forma distinta
    public abstract double calcularSueldo();

    // Comportamiento común: se hereda tal cual
    public void mostrarDatos() {
        System.out.println("Empleado: " + nombre);
        System.out.printf("Sueldo a pagar: $%.0f%n", calcularSueldo());
    }
}
```

EmpleadoFijo.java

```
public class EmpleadoFijo extends Empleado {

    private double bono;

    public EmpleadoFijo(String nombre, double sueldoBase, double bono) {
        super(nombre, sueldoBase);
        this.bono = bono;
    }

    @Override
    public double calcularSueldo() {
        return sueldoBase + bono;
    }
}
```

EmpleadoPorHoras.java

```
public class EmpleadoPorHoras extends Empleado {

    private int horasTrabajadas;
    private double valorHora;

    public EmpleadoPorHoras(String nombre, int horasTrabajadas, double valorHora) {
        super(nombre, 0); // no aplica sueldo base fijo
        this.horasTrabajadas = horasTrabajadas;
        this.valorHora = valorHora;
    }

    @Override
    public double calcularSueldo() {
        return horasTrabajadas * valorHora;
    }
}
```

Main.java

```
public class Main {
    public static void main(String[] args) {
```

```
        Empleado[] empleados = {
            new EmpleadoFijo("Camila Rojas", 650000, 80000),
            new EmpleadoPorHoras("Diego Soto", 120, 4500)
        };

        for (Empleado e : empleados) {
            e.mostrarDatos();
            System.out.println();
        }
    }
}
```

Salida esperada en consola

```
Empleado: Camila Rojas
Sueldo a pagar: $730000

Empleado: Diego Soto
Sueldo a pagar: $540000
```

Explicación de los conceptos aplicados

- El método `mostrarDatos()` nunca necesita saber **cómo** se calcula el sueldo: solo invoca `calcularSueldo()`, y en tiempo de ejecución se ejecuta la versión correcta según el objeto real (polimorfismo).
- Cada subclase está **obligada** a implementar `calcularSueldo()` con `@Override`, porque en la clase abstracta el método no tiene cuerpo.
- Si en el futuro se agrega un nuevo tipo de empleado (por ejemplo, `EmpleadoPorComision`), no es necesario modificar `Empleado` ni `mostrarDatos()`: solo se crea la nueva subclase.

3 Herencia y jerarquías con clases abstractas

Eje 3

Enunciado: Jerarquía de instrumentos musicales

Una escuela de música quiere modelar distintos instrumentos: guitarra, piano y batería. Todos los instrumentos deben poder **afinarse** y **tocarse**, pero cada uno lo hace de forma distinta. Además, todos comparten la acción de **guardarse** en su estuche o funda al finalizar la clase, lo cual es igual para todos.

Se pide: diseñar la clase abstracta `Instrumento` con los atributos y métodos comunes, y tres subclases concretas que hereden de ella.

Diagrama de la jerarquía

```
        Instrumento (abstract)
        ----------------------------
        - nombre : String
        + afinar() : void        [abstracto]
        + tocar() : void          [abstracto]
        + guardar() : void         [concreto]
              |
     ----------------------
     |         |            |
  Guitarra   Piano       Bateria
 (concreta) (concreta)  (concreta)
```

Instrumento.java

```
public abstract class Instrumento {

    protected String nombre;

    public Instrumento(String nombre) {
        this.nombre = nombre;
    }

    public abstract void afinar();
    public abstract void tocar();

    // Comportamiento común heredado por todas las subclases
    public void guardar() {
        System.out.println(nombre + " ha sido guardado en su estuche.");
    }
}
```

Guitarra.java, Piano.java y Bateria.java

```
public class Guitarra extends Instrumento {
    public Guitarra() { super("Guitarra"); }

    @Override
    public void afinar() {
        System.out.println("Afinando las cuerdas de la guitarra.");
    }

    @Override
    public void tocar() {
        System.out.println("Tocando un acorde de guitarra.");
    }
}

public class Piano extends Instrumento {
    public Piano() { super("Piano"); }

    @Override
    public void afinar() {
        System.out.println("Ajustando la tensión de las cuerdas del piano.");
    }

    @Override
    public void tocar() {
        System.out.println("Presionando teclas para interpretar una melodía.");
    }
}

public class Bateria extends Instrumento {
    public Bateria() { super("Batería"); }

    @Override
    public void afinar() {
        System.out.println("Ajustando la tensión de los parches de la batería.");
    }

    @Override
    public void tocar() {
        System.out.println("Tocando un ritmo con baquetas.");
    }
}
```

Main.java

```
public class Main {
    public static void main(String[] args) {
        Instrumento[] instrumentos = {
            new Guitarra(),
            new Piano(),
            new Bateria()
        };

        for (Instrumento instrumento : instrumentos) {
            instrumento.afinar();
            instrumento.tocar();
            instrumento.guardar();
            System.out.println();
        }
    }
}
```

Salida esperada en consola

```
Afinando las cuerdas de la guitarra.
Tocando un acorde de guitarra.
Guitarra ha sido guardado en su estuche.

Ajustando la tensión de las cuerdas del piano.
Presionando teclas para interpretar una melodía.
Piano ha sido guardado en su estuche.

Ajustando la tensión de los parches de la batería.
Tocando un ritmo con baquetas.
Batería ha sido guardado en su estuche.
```

Explicación de los conceptos aplicados

- Las tres subclases **heredan** el atributo `nombre` y el método `guardar()` sin necesidad de reescribirlos.
- `afinar()` y `tocar()` son abstractos porque representan una acción común, pero cada subclase la ejecuta de forma diferente: esta es la esencia de organizar una jerarquía con clases abstractas.
- La clase `Instrumento` no puede instanciarse (`new Instrumento()` generaría un error de compilación), ya que representa un concepto general e incompleto.

4 Patrón de diseño Template Method

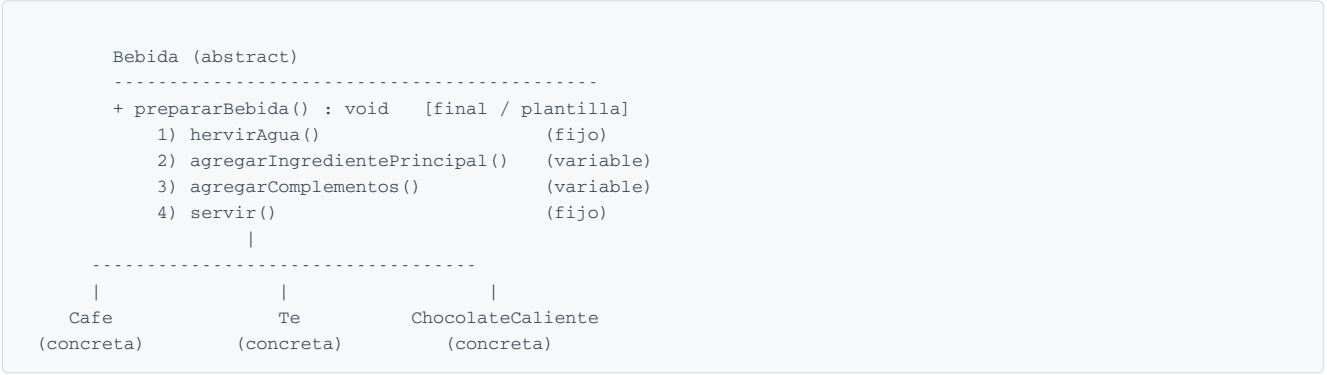
Eje 4

Enunciado: Preparación de bebidas en una cafetería

Una cafetería prepara café, té y chocolate caliente. El **proceso general** de preparación es siempre el mismo y en el mismo orden: hervir agua, agregar el ingrediente principal, agregar los complementos y servir la bebida. Sin embargo, el ingrediente principal y los complementos cambian según la bebida.

Se pide: aplicar el patrón **Template Method** definiendo un método plantilla `final` que fije el orden de los pasos, dejando como abstractos únicamente los pasos que varían.

Diagrama del algoritmo (Template Method)



Bebida.java

```
public abstract class Bebida {

    // Método plantilla: define el orden del algoritmo y no puede modificarse
    public final void prepararBebida() {
        hervirAgua();
        agregarIngredientePrincipal();
        agregarComplementos();
        servir();
    }

    // Paso fijo: igual para todas las bebidas
    private void hervirAgua() {
        System.out.println("Hirviendo agua...");
    }

    // Pasos variables: cada bebida los personaliza
    protected abstract void agregarIngredientePrincipal();
    protected abstract void agregarComplementos();

    // Paso fijo: igual para todas las bebidas
    private void servir() {
        System.out.println("Bebida servida. ¡Disfrútala!\n");
    }
}
```

Cafe.java, Te.java y ChocolateCaliente.java

```
public class Cafe extends Bebida {
    @Override
    protected void agregarIngredientePrincipal() {
        System.out.println("Agregando café molido.");
    }

    @Override
    protected void agregarComplementos() {
        System.out.println("Agregando azúcar y un poco de leche.");
    }
}

public class Te extends Bebida {
    @Override
    protected void agregarIngredientePrincipal() {
        System.out.println("Agregando una bolsita de té.");
    }

    @Override
    protected void agregarComplementos() {
        System.out.println("Agregando miel y una rodaja de limón.");
    }
}

public class ChocolateCaliente extends Bebida {
    @Override
```

```
    protected void agregarIngredientePrincipal() {
        System.out.println("Agregando chocolate en polvo.");
    }

    @Override
    protected void agregarComplementos() {
        System.out.println("Agregando malvaviscos y canela.");
    }
}
```

Main.java

```
public class Main {
    public static void main(String[] args) {
        Bebida[] bebidas = {
            new Cafe(),
            new Te(),
            new ChocolateCaliente()
        };

        for (Bebida bebida : bebidas) {
            bebida.prepararBebida();
        }
    }
}
```

Salida esperada en consola

```
Hirviendo agua...
Agregando café molido.
Agregando azúcar y un poco de leche.
Bebida servida. ¡Disfrútala!

Hirviendo agua...
Agregando una bolsita de té.
Agregando miel y una rodaja de limón.
Bebida servida. ¡Disfrútala!

Hirviendo agua...
Agregando chocolate en polvo.
Agregando malvaviscos y canela.
Bebida servida. ¡Disfrútala!
```

Explicación de los conceptos aplicados

- `prepararBebida()` es `final`: ninguna subclase puede alterar el orden de los pasos del proceso.
- `hervirAgua()` y `servir()` son **privados y concretos**: no cambian entre bebidas, por lo que no tiene sentido que las subclases los sobrescriban.
- `agregarIngredientePrincipal()` y `agregarComplementos()` son los **pasos variables** del algoritmo, y por eso se declaran `abstract`.
- Al llamar a `bebida.prepararBebida()` desde cualquier subclase, siempre se ejecuta la misma secuencia de pasos, cambiando solo el contenido de los pasos variables.

5 Aplicación integradora

Eje 5

Enunciado: Reproductor de una plataforma de streaming

Una plataforma de streaming reproduce distintos tipos de contenido: películas, series y documentales. Al reproducir cualquier contenido, la plataforma siempre sigue el mismo flujo: **cargar el contenido**, **reproducirlo** y **mostrar los créditos al finalizar**. Lo único que cambia entre tipos de contenido es la forma en que se reproduce.

Se pide: combinar clase abstracta, atributos y métodos concretos/abstractos, y el patrón Template Method para modelar el reproductor.

Diagrama de la jerarquía

```
        Contenido (abstract)
        ------------------------------------------
        - titulo : String
        - duracionMinutos : int
        + reproducir() : void        [final / plantilla]
            1) cargarContenido()              (fijo)
            2) reproducirContenido()          (variable)
```

```
            3) mostrarCreditos()               (fijo)
                      |
      ------------------------------------
      |                |                  |
   Pelicula           Serie           Documental
  (concreta)        (concreta)        (concreta)
```

Contenido.java

```
public abstract class Contenido {

    protected String titulo;
    protected int duracionMinutos;

    public Contenido(String titulo, int duracionMinutos) {
        this.titulo = titulo;
        this.duracionMinutos = duracionMinutos;
    }

    // Método plantilla: fija el flujo de reproducción
    public final void reproducir() {
        cargarContenido();
        reproducirContenido();
        mostrarCreditos();
    }

    private void cargarContenido() {
        System.out.println("Cargando \"" + titulo + "\"...");
    }

    // Paso variable: cada tipo de contenido lo implementa
    protected abstract void reproducirContenido();

    private void mostrarCreditos() {
        System.out.println("Fin de \"" + titulo + "\". Mostrando créditos.\n");
    }
}
```

Pelicula.java, Serie.java y Documental.java

```
public class Pelicula extends Contenido {
    public Pelicula(String titulo, int duracionMinutos) {
        super(titulo, duracionMinutos);
    }

    @Override
    protected void reproducirContenido() {
        System.out.println("Reproduciendo película (" + duracionMinutos + " min), sin interrupciones.");
    }
}

public class Serie extends Contenido {
    private int temporada;
    private int episodio;

    public Serie(String titulo, int duracionMinutos, int temporada, int episodio) {
        super(titulo, duracionMinutos);
        this.temporada = temporada;
        this.episodio = episodio;
    }

    @Override
    protected void reproducirContenido() {
        System.out.println("Reproduciendo T" + temporada + "E" + episodio +
                " (" + duracionMinutos + " min).");
    }
}

public class Documental extends Contenido {
    private String tema;

    public Documental(String titulo, int duracionMinutos, String tema) {
        super(titulo, duracionMinutos);
        this.tema = tema;
    }

    @Override
    protected void reproducirContenido() {
        System.out.println("Reproduciendo documental sobre " + tema +
                " (" + duracionMinutos + " min).");
    }
}
```

Main.java

```
public class Main {
    public static void main(String[] args) {
        Contenido[] catalogo = {
            new Pelicula("El viaje del código", 118),
            new Serie("Bugs y Soluciones", 42, 2, 5),
            new Documental("El mundo de Java", 55, "lenguajes de programación")
```

```
        };

        for (Contenido contenido : catalogo) {
            contenido.reproducir();
        }
    }
}
```

Salida esperada en consola

```
Cargando "El viaje del código"...
Reproduciendo película (118 min), sin interrupciones.
Fin de "El viaje del código". Mostrando créditos.

Cargando "Bugs y Soluciones"...
Reproduciendo T2E5 (42 min).
Fin de "Bugs y Soluciones". Mostrando créditos.

Cargando "El mundo de Java"...
Reproduciendo documental sobre lenguajes de programación (55 min).
Fin de "El mundo de Java". Mostrando créditos.
```

Explicación de los conceptos aplicados

- Este ejercicio combina **todo lo visto en la semana 2**: atributos comunes en la clase abstracta, un método concreto reutilizado (`cargarContenido()` y `mostrarCreditos()`), un paso variable declarado como abstracto (`reproducirContenido()`) y el patrón Template Method aplicado mediante el método `final reproducir()`.
- Cada subclase añade sus propios atributos específicos (por ejemplo, `temporada` y `episodio` en `Serie`), demostrando que la herencia no impide que cada clase concreta tenga características propias.
- Si mañana se agrega un nuevo tipo de contenido, como `Podcast`, basta con crear una nueva subclase que implemente `reproducirContenido()`: no es necesario tocar `Contenido` ni el código que recorre el catálogo.

Resumen de conceptos clave

Concepto	¿Qué es?	Ejemplo en esta guía
Clase abstracta	Clase que no se puede instanciar; actúa como plantilla base.	`FiguraGeometrica`, `Empleado`, `Instrumento`, `Bebida`, `Contenido`
Método abstracto	Declarado sin cuerpo; obliga a las subclases a implementarlo.	`calcularArea()`, `calcularSueldo()`, `tocar()`, `agregarComplementos()`
Método concreto	Tiene implementación en la clase abstracta y puede reutilizarse o sobrescribirse.	`mostrarInformacion()`, `mostrarDatos()`, `guardar()`
Herencia	Las subclases obtienen atributos y métodos no privados de la clase base.	Todas las subclases de cada ejercicio
Template Method	Un método `final` define el orden fijo de un algoritmo, delegando los pasos variables a métodos abstractos.	`prepararBebida()`, `reproducir()`

Sugerencia de uso: pide a tus estudiantes que primero intenten resolver cada enunciado por su cuenta antes de revisar la solución, y que luego modifiquen el código agregando una subclase nueva (por ejemplo, un `Cuadrado`, un `EmpleadoPorComision` o un `Podcast`) para verificar que comprendieron la extensibilidad de la jerarquía.